

HorRAGor BOT



Introduction & Objectifs

L'objectif de ce projet est de concevoir **HorRAGor** (ou GorRAGor), un agent conversationnel spécialisé dans l'univers de l'horreur (cinéma, littérature, jeux vidéo). Pour que cet agent soit pertinent et évite les hallucinations, il doit s'appuyer sur une base de connaissances hybride, riche et parfaitement structurée.

Après avoir industrialisé l'ingestion et la réconciliation de nos données de l'horreur dans la phase précédente, l'objectif de cette Partie 2 est de concevoir le "cerveau" de notre application. Vous allez développer un agent conversationnel autonome (ou Agentic AI) basé sur l'architecture ReAct (Reason + Act).

Le HorRAGor BOT devra être capable de raisonner, d'interroger intelligemment son savoir structuré et vectoriel, de calculer l'âge des œuvres, de recommander des films similaires et, si nécessaire, d'aller chercher des informations approfondies "en direct" sur le web.

SOMMAIRE

Introduction & Objectifs.....	0
L'orchestration avec Prefect (Option).....	2
Travail partagé.....	2
L'Interface Utilisateur (Front-End) : Streamlit ou Gradio.....	2
Fichier de Configuration .streamlit/config.toml : Personnalisation	
Graphique.....	3
Exemple:.....	3
Le Composant Back-End : L'API FastAPI.....	4
Architecture de la Mémoire Éphémère (FAISS).....	4
La Boîte à Outils de l'Agent (Tools Spécifiques).....	5

L'orchestration avec Prefect (Option)

C'est ici que tout s'assemble. La Partie 1 demandait de lancer 5 sources en parallèle. Au lieu d'un script Python global un peu fragile, vous allez devoir utiliser Prefect pour orchestrer le flux. Le DAG HorRAGor :

- **Task 1 à 4** : Lancement en parallèle des tâches d'extraction (API TMDB, Scraping Rotten Tomatoes, BDD, etc.).
- **Task 5** : Un spark-submit envoyé sur le cluster pour traiter les gros volumes textuels.
- **Task 6** (Dépendance) : Une fois que toutes les extractions sont OK, une tâche de réconciliation (Fuzzy Matching) fusionne le tout.
- **Task 7** : Persistance finale dans Supabase.

Travail partagé

L'Interface Utilisateur (Front-End) : Streamlit ou Gradio

Pour rendre le moteur de l'agent accessible, vous devez développer une interface web de chat dynamique en utilisant au choix Streamlit ou Gradio. Cette interface servira de point d'entrée unique pour l'utilisateur final et devra respecter les exigences fonctionnelles suivantes :

- **Composants de Chat Dédiés** : Intégration des éléments natifs (comme `st.chat_input` et `st.chat_message` pour Streamlit) pour afficher un historique de discussion propre sous forme de bulles de conversation.
- **Indicateur d'Activité (Loader) option** : Mise en place d'un retour visuel animé pendant la phase de réflexion de l'agent pour garantir une expérience utilisateur fluide et éviter les doubles soumissions de questions.
- **Découplage Strict de l'Architecture** : L'interface ne doit contenir aucune logique métier liée au LLM, à LangGraph ou à Supabase. Elle a pour rôle unique de capturer la question, de l'envoyer via une requête HTTP asynchrone à l'API FastAPI, puis d'afficher la réponse JSON reçue.

Fichier de Configuration `.streamlit/config.toml` : Personnalisation Graphique

Afin d'aligner l'interface utilisateur avec l'identité visuelle sombre et thématique du **HorRAGor BOT**, vous devez configurer le fichier `.streamlit/config.toml` à la racine de votre dossier de Front-End. Ce fichier permet d'imposer un thème graphique cohérent dès le chargement de l'application, évitant ainsi le thème clair par défaut.

Ce fichier doit obligatoirement être poussé sur le dépôt Git pour garantir que l'application s'exécute avec le bon thème chez tous les utilisateurs.

Exemple:

```
[theme]

# Couleurs principales de l'univers "Chat Horror"

# Rouge sang pour les boutons et les éléments actifs
primaryColor = "#8b0000"

# Noir profond pour l'arrière-plan de l'application
backgroundColor = "#111111"

# Gris très sombre pour la barre latérale et les bulles de chat
secondaryBackgroundColor = "#1b1f24"

# Blanc cassé / Gris clair pour une lecture confortable
textColor = "#e2e8f0"

font = "sans serif"
```

Le Composant Back-End : L'API FastAPI

Afin d'industrialiser le projet et de séparer strictement la logique métier de l'interface utilisateur, vous devez développer une API REST asynchrone à l'aide de FastAPI. Cette API agira comme le chef d'orchestre unique du Back-End et devra répondre aux spécifications suivantes :

- **Endpoint unique de Chat (POST /chat)** : Vous créerez une route sécurisée qui réceptionne la question de l'utilisateur, initialise le graphe d'état LangGraph, pilote l'exécution de l'agent (et de ses outils) jusqu'au verdict final du Juge, puis retourne la réponse validée au format JSON.
- **Typage strict avec Pydantic** : Les structures des données entrantes (requêtes utilisateur) et sortantes (réponses de l'agent, verdict du nœud d'évaluation) doivent être rigoureusement modélisées et validées à l'aide de classes Pydantic.
- **Encapsulation des accès Supabase** : Pour des raisons évidentes de sécurité, toutes les clés d'API et l'initialisation du client SDK Supabase devront être confinées au sein de ce composant Back-End. Aucune communication directe avec la base de données ne doit être initiée depuis l'interface client.

Architecture de la Mémoire Éphémère (FAISS)

Pour optimiser les performances de l'agent et éviter de saturer la base de données principale, vous mettrez en place un index FAISS local (en RAM) :

- **Rôle** : Cet index contiendra uniquement les couples [Nom du film : ID].
- **Usage** : Avant toute action lourde, l'agent utilisera cet index ultra-rapide comme un "routeur" pour valider l'existence d'un film et récupérer son identifiant unique de référence.

La Boîte à Outils de l'Agent (Tools Spécifiques)

Pour interagir avec le monde réel et ses connaissances, votre LLM (via LangChain ou LlamaIndex) aura accès exclusivement aux outils suivants :

- **Tool 1** : Requêtes SQL Spécialisées (**query_movie_metadata**)
 - Sécurité : Le LLM ne doit jamais générer de requêtes SQL brutes. Vous écrirez des fonctions Python saines exécutant des requêtes précises.
 - Rôle : Permettre à l'utilisateur de discuter de l'œuvre et d'obtenir les infos de base (le réalisateur, l'année, le genre, la note moyenne, le casting).
- **Tool 2** : Le Recommandeur de Films Similaires (**find_similar_horror_movies**)
 - Rôle : Aider l'utilisateur à découvrir de nouvelles pépites.
 - Technique : Cet outil exploitera l'extension vectorielle de Supabase (pgvector). En lui passant l'ID d'un film, il effectuera une recherche par similarité cosinus sur les vecteurs de synopsis de la Partie 1 pour retourner les \$k\$ films les plus proches sémantiquement.
- **Tool 3** : Scraping On-Demand (**scrape_detailed_synopsis**)
 - Rôle : Un bonus activé uniquement si l'utilisateur demande des détails profonds ou des anecdotes non présents en base.
 - Technique : L'agent utilise Selenium/Requests pour cibler la page Wikipédia ou équivalente du film, extrait le synopsis complet et le résume pour répondre au besoin.
- **Tool 4** : Le Calculateur Temporel (**calculate_movie_age**)
 - Rôle : Une fonction Python native qui prend l'année de sortie du film et calcule son âge exact par rapport à l'année actuelle.
- **Tool 5 (Optionnel)** : Le Simulateur de Survie (**horror_survival_simulator**)
 - Rôle : Un outil ludique où le LLM utilise le synopsis détaillé du film pour simuler les chances de survie de l'utilisateur dans ce scénario.

